SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Content

1. Introduction....3
1.1. Purpose.... 3
1. Architecture:.... 3
2. Data Flows:....3
3. Model Process:.... 3
1.2. Background....3
1.3. Target Stakeholder.... 4
2. System Architecture & Design....5
2.1. High Level Architecture.... 5
2.1.1. Overview.... 5
2.1.2 LLM as Service Layer.... 5
2.1.3 Dependency Diagram.... 5
2.1.4 Sequence Diagram.... 6
2.2. Technological Stack....7
2.2.1. Frontend....7
Next.js 15 (App Router) + React 19 + Tailwind CSS – Ensures rapid Vibe Coding, Server Components for performance, and a responsive, industrial-tech data grid aesthetic.... 7
2.2.2. Backend.... 7
Next.js Route Handlers – Acts as the RESTful Backend-for-Frontend (BFF) to securely handle Firebase Admin SDK operations and GLM API communications....7
2.2.3. Database.... 7
Firebase Cloud Firestore (NoSQL) & Firebase Auth – Denormalized document structure to minimize client-side joins. Hybrid auth approach using Firebase Custom Claims + Next.js Middleware for route protection.... 7
2.2.4. Cloud/Deployment.... 7
Vercel – Seamless edge deployment for the Next.js application, ensuring low latency and high availability for serverless functions.... 7
2.3. Key Data Flows.... 8
2.3.1. Data Flow Diagram (DFD)....8
2.3.2. Normalized Database Schema....9
3. Functional Requirements & Scope....10
3.1. Minimum Viable Product.... 10

3.2. Non-Functional Requirements (NFRs)..... 11
3.3. Out of Scope / Future Enhancements..... 11
4. Monitor, Evaluation, Assumptions & Dependencies..... 12
4.1. Technical Evaluation..... 12
4.1.1. Prompt Tuning & A/B Testing..... 12
4.1.2. Emergency Rollback (ER)..... 12
4.1.3. Priority Matrix..... 12
4.2. Monitoring..... 12
4.3. Assumptions..... 12
4.4. External Dependencies..... 12
5. Project Management & Team Contributions..... 13
5.1. Project Timeline..... 13
5.2. Team Members Role..... 15
5.3. Recommendations..... 15

1. Introduction

FutureJob is a stateful, adaptive workflow engine utilizing Z.AI's GLM (General Language Model) as the central reasoning orchestrator. It transforms fragmented, unstructured recruitment processes into intelligent, automated career routing, bypassing traditional keyword matching Applicant Tracking Systems (ATS).

1.1. Purpose

The system analysis document highlights the technical scope and design-related decisions behind the FutureJobSenpai development.

So, the key element of system that has been covered are as followed:

1. Architecture:
    a. A modern Serverless Next.js (App Router) web application utilizing a Backend-for-Frontend (BFF) pattern. Deployed on Vercel with data persisted in Firebase Cloud Firestore (NoSQL).
2. Data Flows:
    a. How raw unstructured resume text flows from a student's client, gets parsed by the GLM into a strict JSON profile, gets matched against company job requirements, and finally triggers automated applications or AI-generated "Skill Quests"
3. Model Process:
    a. The AI model operates as a stateful orchestrator. It executes multi-step reasoning: structuring data, scoring bidirectionally, generating reasoning traces, and routing states (e.g., generating actionable feedback loops for rejected applications)
4. Role of Reference:
    a. This document is a reference for all developers and testers to understand the high-level architecture, strict AI schema handling, and database conventions of this project

1.2. Background

Traditional recruitment relies heavily on simple keyword-matching ATS systems. These systems often fail to understand the semantic context of a candidate's experience, leading to qualified candidates being ignored and companies receiving poorly matched applicants.

Changes in Major Architectural Components:

FutureJobSenpai replaces the keyword-matching layer with a contextual reasoning engine powered by Z.AI's GLM. Instead of just filtering, the system acts autonomously—auto-applying for high matches (>85%) and generating "Skill Quests" (actionable feedback loops) for rejections.

1.3. Target Stakeholder

Stakeholders	**Roles**	**Expectations**
Student (Candidate)	Submits unstructured text/resume, tracks application states, and completes Skill Quests.	Seamless onboarding, automated applications to relevant jobs, and constructive, actionable feedback upon rejection.
Company (Employer)	Inputs natural language job requirements and reviews pre screened candidates.	High-quality candidate matching with transparent "reasoning traces" and an efficient verification and screening process.
Admin	Manages platform integrity and verifies legitimate companies.	Secure dashboard to toggle company verification statuses (isVerified) before they can post jobs.
Development Team	Builds the Next.js platform and maintains Firebase / AI integrations.	Clear API usage contracts, strict JSON schemas for GLM outputs, and robust fallback/retry mechanisms.
QA Team	Validates the AI reasoning stability and UI/UX flows.	Assured handling of edge cases (e.g., malformed AI responses) and correct Firestore security rule enforcement.

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview

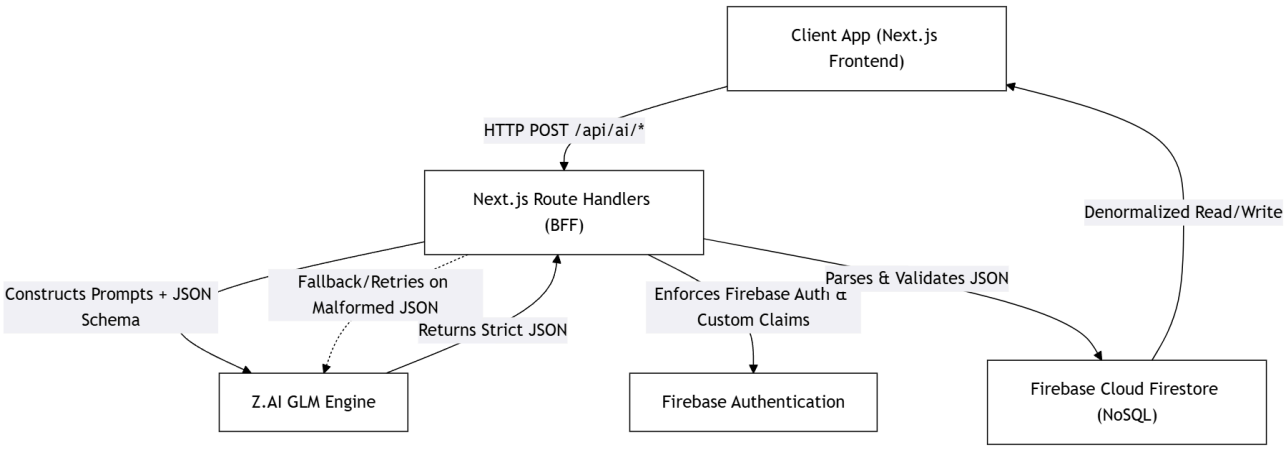
Type	Details
System	Responsive Web Application
Architecture	Serverless Backend-for-Frontend (BFF) with Cloud NoSQL

FutureJob is structured as a client-server-based system interacting with three main client roles: Students, Companies, and Admins. Access is strictly managed via Firebase Custom Claims and Next.js Middleware. The Next.js Route Handlers act as the BFF layer, communicating directly with the Z.AI GLM engine and persisting denormalized data in Cloud Firestore.

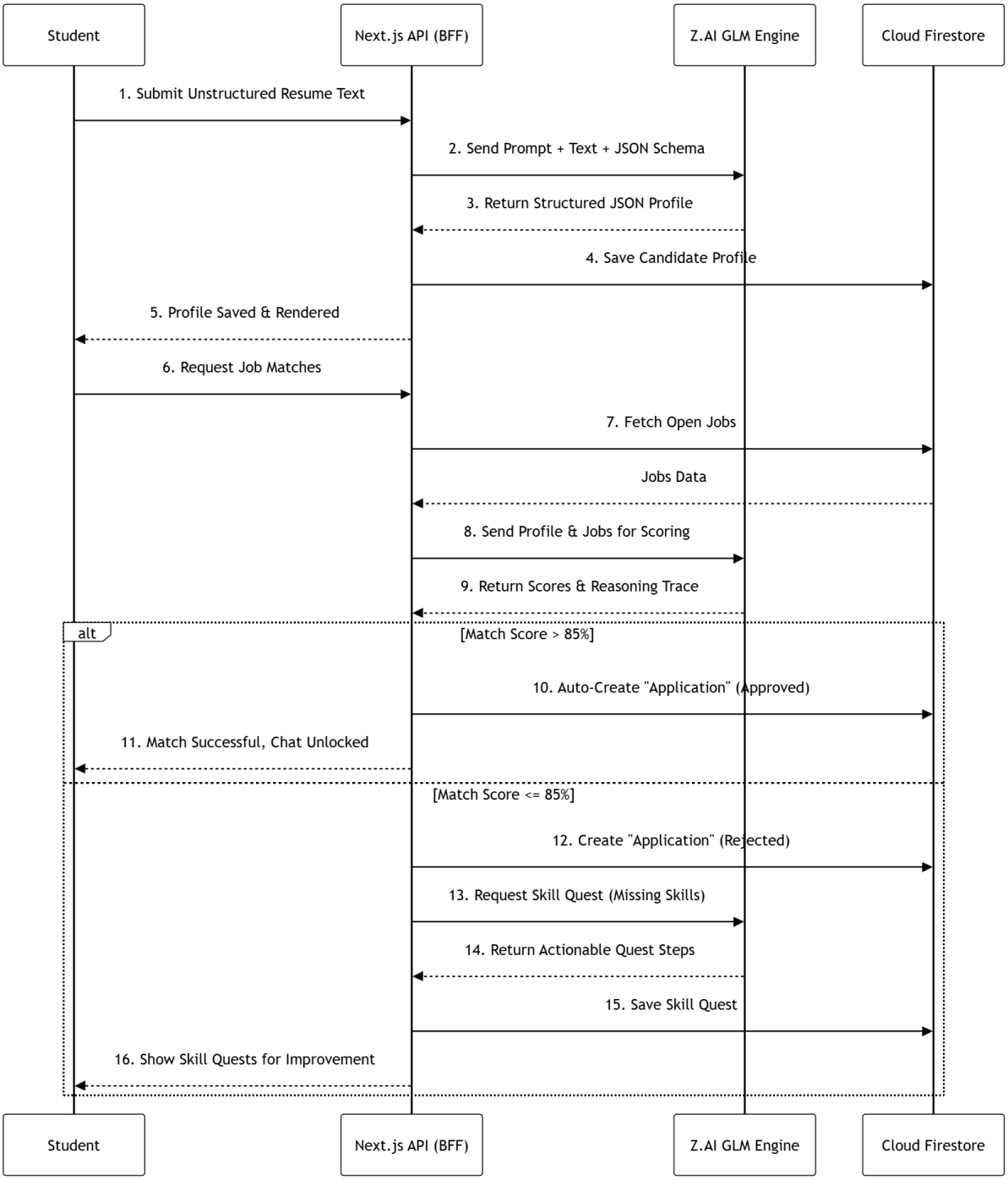
2.1.2 LLM as Service Layer

The architecture of your product needs to show how GLM is integrated as a service layer- not limited to a generic “AI Module”. The following Dependency diagram includes:

2.1.3 Dependency Diagram



2.1.4 Sequence Diagram



2.2. Technological Stack

2.2.1. Frontend

Next.js 15 (App Router) + React 19 + Tailwind CSS – Ensures rapid Vibe Coding, Server Components for performance, and a responsive, industrial-tech data grid aesthetic.

2.2.2. Backend

Next.js Route Handlers – Acts as the RESTful Backend-for-Frontend (BFF) to securely handle Firebase Admin SDK operations and GLM API communications.

2.2.3. Database

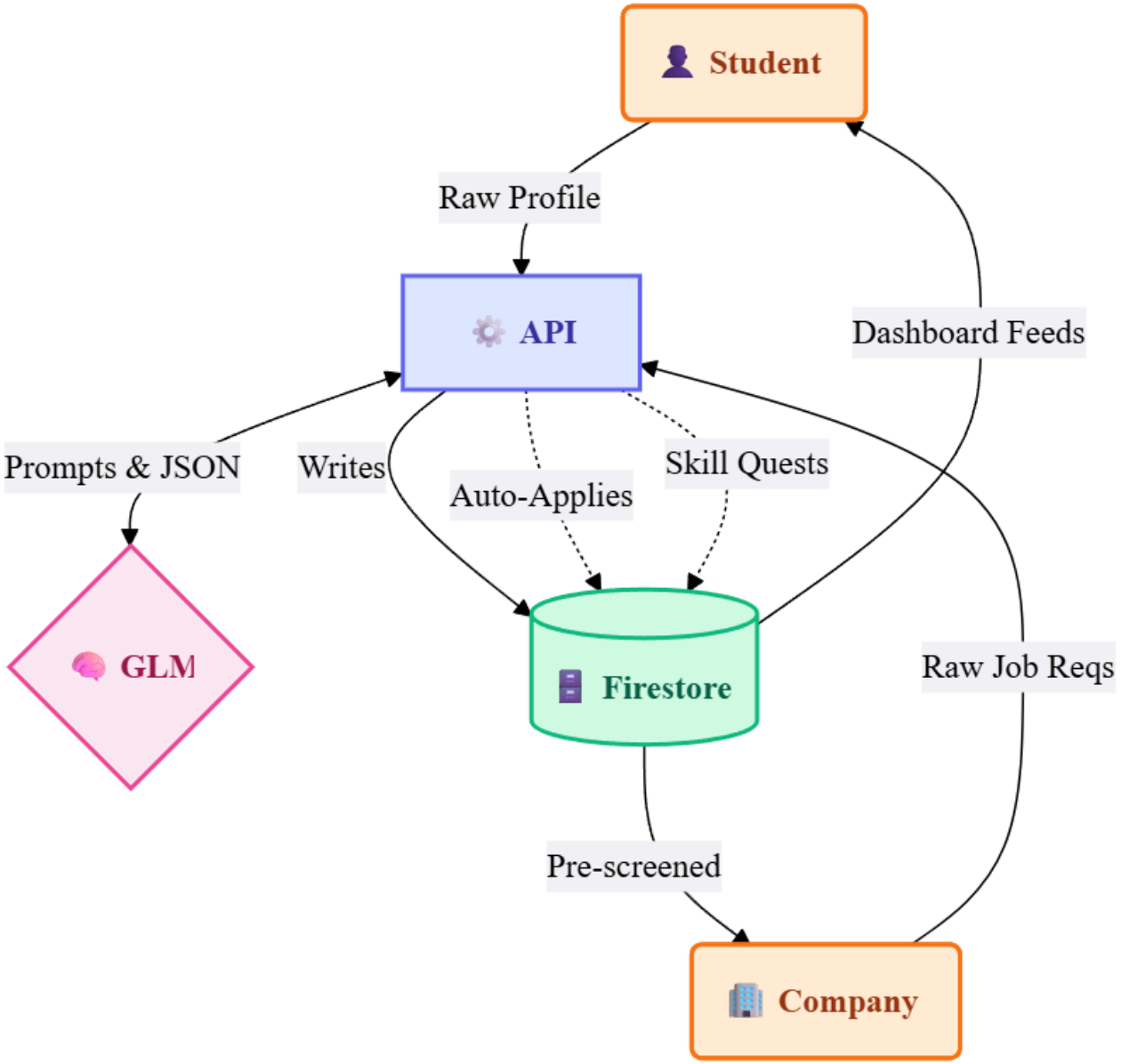
Firebase Cloud Firestore (NoSQL) & Firebase Auth – Denormalized document structure to minimize client-side joins. Hybrid auth approach using Firebase Custom Claims + Next.js Middleware for route protection.

2.2.4. Cloud/Deployment

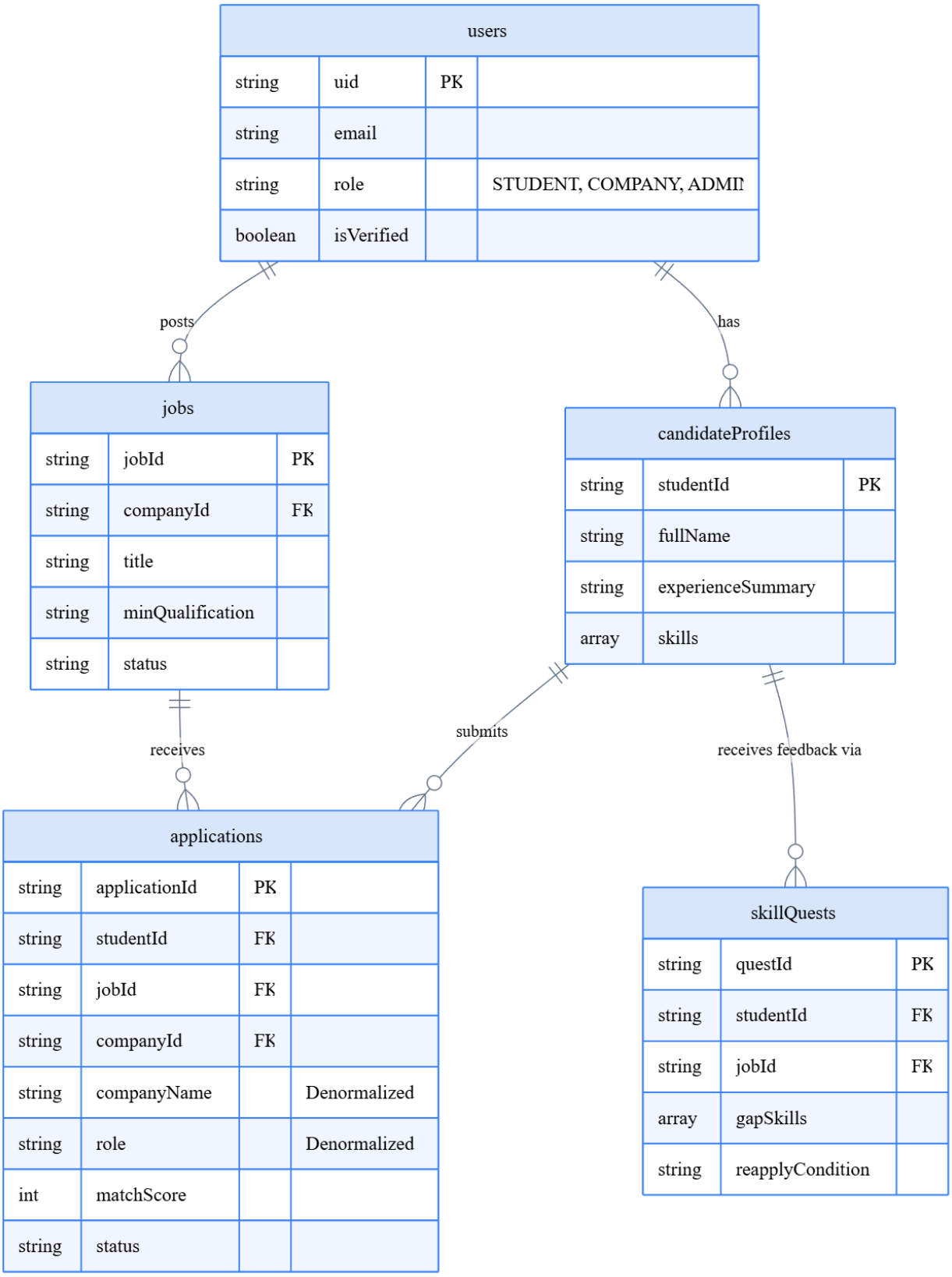
Vercel – Seamless edge deployment for the Next.js application, ensuring low latency and high availability for serverless functions.

2.3. Key Data Flows

2.3.1. Data Flow Diagram (DFD)



2.3.2. Normalized Database Schema



3. Functional Requirements & Scope

3.1. Minimum Viable Product

#	Features	Description
1	AI Profile Builder	Converts raw, unstructured student input into a structured, optimized JSON profile via GLM.
2	Natural Language Job Posting	Companies input conversational job requirements; GLM structures them into precise data points.
3	Bidirectional AI Matching	GLM calculates match scores. Scores > 85% trigger automated application submissions.
4	Candidate Screening Dashboard	Companies view pre-filtered applicants sorted by GLM match score and transparent reasoning traces.
5	Actionable Skill Quests	Rejected candidates receive a GLM-generated, multi-step feedback loop based on missing requirements.

3.2. Non-Functional Requirements (NFRs)

Quality (e.g.)	**Requirements (e.g.)**	**Implementation (e.g)**
Scalability	Must handle parallel AI processing requests during peak traffic.	Utilize Vercel's serverless infrastructure. Move heavy background matching tasks to Cloud Functions if needed.
Reliability	AI JSON responses must be strictly formatted; system shouldn't crash on hallucinations.	Implement try/catch blocks with up to 3 silent retries in API routes before failing gracefully.
Performance	Dashboard views must load instantly without excessive database querying.	Enforce NoSQL denormalization (embedding names/scores in application docs) to minimize read operations.
Security	Companies must not post jobs without authorization.	Hybrid Auth: Admin manually toggles isVerified: true . Next.js Middleware blocks unverified writes.

3.3. Out of Scope / Future Enhancements

a. Built-in WebRTC Video Interviewing platform.
b. Gamified 3D virtual environment for completing Skill Quests.
c. Deep integrations with third-party HR Payroll and ERP systems.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation

4.1.1. Prompt Tuning & A/B Testing

Different GLM prompt templates for the matching engine will be evaluated based on the acceptance rate of the AI's recommendations by the Company users.

4.1.2. Emergency Rollback (ER)

If the GLM API experiences sustained outages, the system will gracefully degrade. The automatic application and AI matching features will be temporarily disabled, allowing Students and Companies to fall back to a manual application and screening process.

4.1.3. Priority Matrix

- P1 Critical: Firebase Auth or Firestore down -> Immediate communication to users, await vendor resolution.
- P1 Critical: GLM JSON malformation rate > 10% -> Rollback to previous prompt template version.
- P2 High: Index building errors in Firestore -> Implement client-side fallback sorting (e.g., skillQuests memory sorting).

4.2. Monitoring

- Vercel Analytics & Logs: Used to monitor API Route latencies and monitor the success/retry rate of GLM parsing functions.
- Firebase Console: Monitor Firestore read/write quotas and ensure index health.

4.3. Assumptions

1. Z.AI's GLM API remains highly available and consistently adheres to requested output schemas. application/json
2. Modern browsers are used by the target stakeholders (leveraging React 19 / Next.js features).

4.4. External Dependencies

Tools	Purpose	Risks

Z Ai GLM API	Central reasoning orchestrator for structuring data, matching, and quest generation.	Rate limits or malformed output. Mitigation: Implement 3x retry logic and strict JSON parsing fallbacks.
Firebase Services	Auth, Firestore Database, and Storage.	Cold starts or missing composite indexes. Mitigation: Pre-define indexes in firestore.indexes.json and utilize BFF pattern to abstract DB logic.

5. Project Management & Team Contributions

5.1. Project Timeline

- Day 1-2: Ideation, Next.js Setup, Firebase config, architecture planning.
- Day 3-5: GLM API integration, Prompt Engineering, Schema validation.
- Day 6-8: Core UI implementation (Student & Company dashboards).
- Day 9-10: System Integration, Error handling (Retries/Fallbacks), final deployment & testing

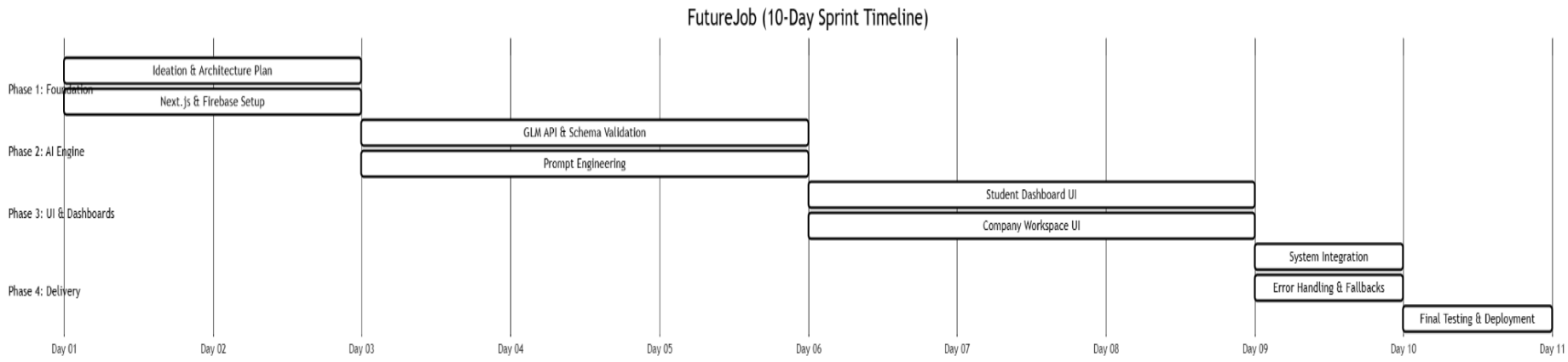


Figure 5.1: Gantt Chart for Project Timeline

5.2. Team Members Role

Team Member	Responsibilities
Koo Wei Gang	Core Setup & Infrastructure: Next.js setup, Middleware, Firebase Auth/Firestore, AI Engine Integration (GLM routing, schemas, rate limits), Admin Verification.
New Wei Jun	Student Experience (UI & Backend): Student Dashboard, AI Profile Builder UI/Logic, Application Status Tracker, Job matching view, Skill Quest rendering.
Tan Choon Da	Company Experience (UI & Backend): Company Workspace Dashboard, Natural Language Job Posting UI, Chat Platform, Automated Notifications.

5.3. Recommendations

- Data Optimization: As the user base grows, implement caching (e.g., Redis or Next.js built-in cache) for active Job Postings to reduce Firestore document reads and lower costs.
- AI Cost Management: Cache identical AI query results (e.g., matching the exact same profile to the exact same job) to prevent redundant GLM API calls.